E-Commerce BI Pipeline

GCP Data & AI Engineer Assessment

End-to-End Pipeline — Theory, Design & Production Considerations

1. Executive Summary

Modern data-driven organisations depend on reliable, timely, and accessible data pipelines. Without automation, even a small lapse in manual processes can cascade into business decisions made on stale or incorrect data. This document describes a fully automated, cloud-native data pipeline built on Google Cloud Platform that addresses three interconnected data engineering problems faced by an e-commerce company operating five online stores across the United States.

The pipeline is designed around three core principles: reliability through automation, trust through data quality enforcement, and accessibility through natural language interfaces. Each layer of the architecture is independently deployable, observable, and replaceable — consistent with modern microservices thinking applied to data infrastructure.

Business Problem	Engineering Solution	GCP Service
Manual daily CSV uploads — no automation, single point of human failure	Event-driven ingestion pipeline triggered by file arrival in cloud storage	Cloud Storage + Eventarc + Cloud Functions
Dirty data reaching dashboards — duplicates, bad dates, inconsistent names	Automated validation, cleaning, and quarantine pipeline running daily before dashboards refresh	BigQuery Stored Procedure + Scheduled Query
Non-technical users waiting on BI team for every question	Natural language data agent connected directly to the warehouse	BigQuery Data Agent powered by Gemini

2. Proof of Concept Architecture

2.1 Architectural Philosophy

The pipeline follows a medallion architecture — a layered data design pattern increasingly standard in modern cloud data warehouses. Data flows through progressively higher quality zones, with each layer serving a distinct purpose:

- Raw layer — immutable landing zone. Exact copy of source data, never modified. Serves as the audit record and re-processing source.
- Gold layer — curated, business-ready data. Typed, validated, cleaned, deduplicated. The single source of truth for all dashboards and queries.
- Consumption layer — natural language interface. Business users access gold data through a Gemini-powered agent without writing SQL.

This separation of concerns means failures in one layer do not contaminate others. A malformed source file will pollute only the raw layer — the gold layer remains untouched until the file is corrected and the pipeline re-run.

2.2 POC Architecture Diagram

The following diagram represents exactly what was built and deployed on GCP Free Tier for this assessment. Each numbered step corresponds to a discrete component described in the sections that follow.

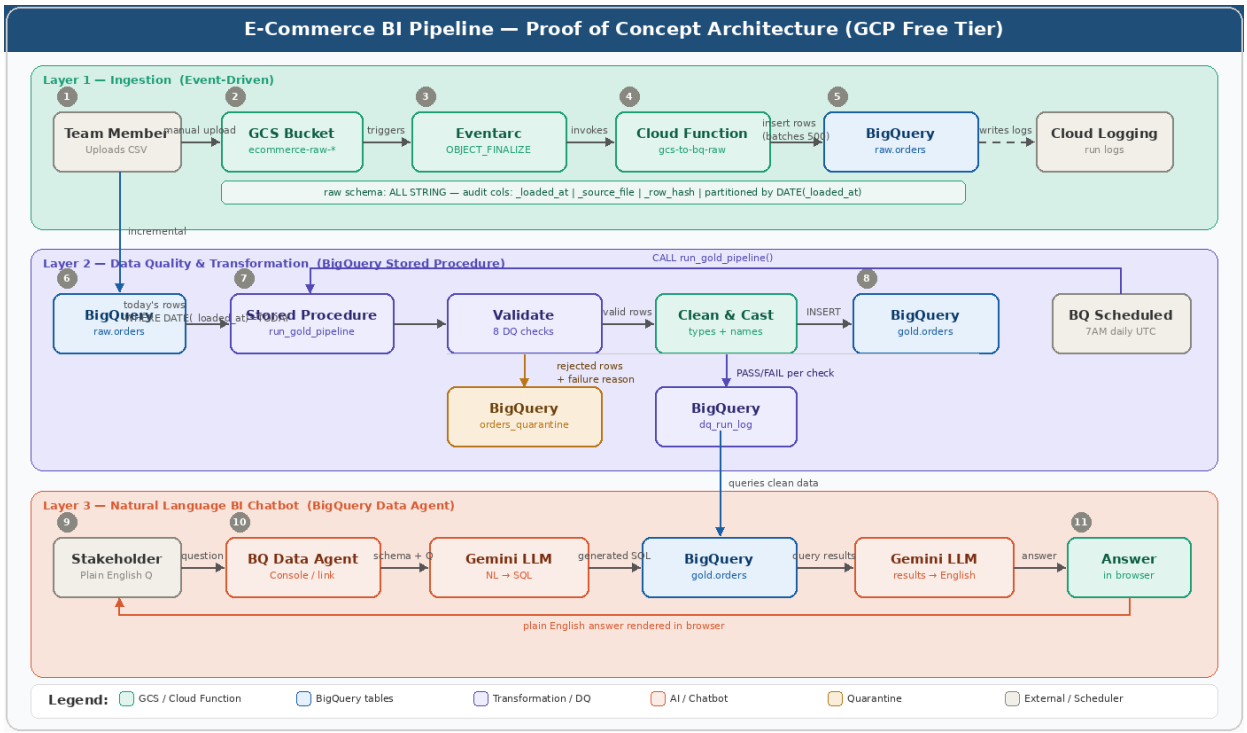

Figure 1 — Proof of Concept pipeline as deployed on GCP Free Tier

2.3 Step-by-Step Flow

Step	Component	What Happens
1	Team Member	CSV file of daily orders is manually uploaded to GCS bucket
2	GCS Bucket	File lands in gs://ecommerce-raw-orders-hassan/date/orders.csv
3	Eventarc	OBJECT_FINALIZE event fires automatically on file arrival
4	Cloud Function	gcs-to-bigquery-raw reads CSV, appends audit cols, inserts to BQ in batches of 500
5	BigQuery raw.orders	All 10,000 rows land as STRING — fully typed later in gold layer
6	BQ Scheduled Query	At 7AM UTC, CALL run_gold_pipeline() is invoked automatically
7	Stored Procedure	Reads today's rows (DATE(_loaded_at) = CURRENT_DATE())
8	BigQuery gold.orders	Clean rows inserted — typed, normalised, validated
8b	orders_quarantine	Rejected rows stored with _failure_reason column
8c	dq_run_log	One PASS/FAIL row per check written for observability
9	Stakeholder	Opens BigQuery Data Agent link in browser, types question
10	BQ Data Agent + Gemini	Question → SQL generated → query executed → result summarised
11	Answer	Plain English answer returned in browser within seconds

Layer	Table / Service	Record Count (test data)
Raw	raw.orders	10,000 rows — 100% of source loaded
Gold — clean	gold.orders	~8,900 rows — passed all DQ checks
Gold — rejected	gold.orders_quarantine	~1,100 rows — failed one or more checks
Gold — log	gold.dq_run_log	7 rows — one per check per daily run

3. Layer 1 — Automated Ingestion Pipeline

3.1 Theory & Design Rationale

The ingestion layer solves a fundamental reliability problem: a manual process that depends on a single person performing a task every morning. This is an anti-pattern in production data engineering because it introduces a human single point of failure, offers no retry mechanism, produces no audit trail, and scales only as fast as a person can work.

The solution replaces this with an event-driven architecture — a design pattern where components react to state changes rather than being explicitly invoked on a schedule. In event-driven ingestion, the pipeline does not poll for new files or wait for a scheduled time. It listens. The moment a file arrives in storage, the infrastructure reacts. This approach minimises latency between data availability and data accessibility, and eliminates the dependency on any individual.

3.2 Component Design Decisions

Why Google Cloud Storage as the landing zone

GCS decouples the ingestion trigger from the source system. The source system deposits a file using a simple upload — it needs no knowledge of BigQuery, Cloud Functions, or any downstream processing. This loose coupling is a core principle of robust data pipeline design. It also provides an immutable copy of the original file for re-processing if bugs are discovered downstream.

Why Cloud Functions over Cloud Run or Dataflow

For a daily file of 5,000 to 50,000 rows, a Cloud Function is the optimal compute choice. Cloud Functions are serverless, bill only for execution time (milliseconds to seconds for this workload), require no infrastructure management, and cold-start within 500ms. Cloud Dataflow — Google's managed Apache Beam service — is architecturally superior for streaming or very large batch workloads, but its overhead makes it economically unjustifiable for a daily 10,000-row file. The right tool is matched to the scale of the problem.

Why all columns are STRING in the raw table

The raw table follows the principle of load-first, validate-later. Every column in raw.orders is defined as STRING regardless of its true type. A malformed date value like '2025-99-99' would cause a typed load job to fail, potentially losing the entire batch. By accepting all values as strings, the raw layer guarantees 100% load success. Type casting and validation happens in the transformation layer where failures are handled gracefully rather than catastrophically.

Why partitioning by load date

BigQuery charges based on bytes scanned per query. Without partitioning, a query asking 'what was loaded today?' would scan every row in the table — three years of history. With DATE(_loaded_at) partitioning, the same query scans only today's partition, reducing cost and latency by 95%+. This design decision compounds in value as the table grows.

3.3 Audit Columns

Column	Type	Purpose
_loaded_at	TIMESTAMP	Exact UTC timestamp of load. Used by gold pipeline to filter today's rows. Enables incremental processing without re-scanning history.
_source_file	STRING	Full GCS path (gs://bucket/date/file.csv). Every row in gold can be traced back to the exact file it came from.
_row_hash	STRING	MD5 hash of all field values. Detects accidental re-uploads of the same file. If hash exists in gold or quarantine, row is skipped.

4. Layer 2 — Data Quality Pipeline

4.1 Theory & Design Rationale

Data quality is not a feature — it is a foundation. A data warehouse that cannot be trusted is worse than no data warehouse at all, because it creates false confidence. The marketing team's loss of trust in dashboards is a symptom of a deeper architectural gap: there was no systematic, automated enforcement of data contracts between the source system and the consuming dashboards.

A data contract is a formal agreement about what data a producer will deliver to a consumer. It specifies expected types, value ranges, uniqueness constraints, and referential integrity rules. When contracts are enforced manually through ad hoc queries, they are inevitably inconsistent, delayed, and fragile. This pipeline replaces manual checking with a declarative, automated enforcement layer that runs before any dashboard can display the data.

> **The Core Principle: Fail Fast, Fail Loudly**
>
> Bad data should be caught as early as possible in the pipeline — at the transformation stage, not after it has been served to a dashboard. Every rejected row is quarantined with an explicit, human-readable reason. Nothing is silently dropped. Nothing is silently corrected without a record.

4.2 The Three-Stage Transformation Model

1. Validate — every row is evaluated against 8 DQ checks. Failures are collected as a comma-separated string. A row can fail multiple checks simultaneously.
2. Clean — rows that pass validation have transformations applied: type casting from STRING to typed columns, INITCAP normalisation of product names, canonical store name mapping, NULL coalescing.
3. Route — clean rows insert into gold.orders. Failed rows insert into gold.orders_quarantine with failure reason. A summary log entry is written per check to gold.dq_run_log.

4.3 The 8 Data Quality Checks

Check Name	Problem Class	Why It Matters
Missing order_id	Referential integrity	Without a primary key, records cannot be tracked, updated, or joined to other tables
Duplicate order_id in batch	Uniqueness constraint	Duplicate keys corrupt aggregations and revenue calculations downstream
Unparseable order_date	Type validity	A date that cannot be parsed cannot be used for any time-based analysis

Check Name	Problem Class	Why It Matters
Future order_date	Business logic violation	Orders cannot be placed in the future — indicates a data entry error or system clock issue
Ship date before order date	Temporal consistency	A physical impossibility — signals corrupted or swapped field values in the source
Zero revenue with quantity > 0	Numeric consistency	Revenue of zero with positive quantity indicates a pricing system failure or ETL bug
Missing store_name	Completeness	Store is a core dimension — reports segmented by store cannot include this record
Missing order_date string	Completeness	The most critical business dimension — a record without a date cannot be used in any time series analysis

4.4 Incremental Processing

The pipeline processes only rows loaded today (WHERE DATE(_loaded_at) = CURRENT_DATE()). On a table holding three years of data, a full scan would process millions of rows daily when only thousands are new. Incremental processing means the pipeline's runtime and cost remain constant regardless of historical data volume.

4.5 Quarantine as an Audit Mechanism

- Root cause analysis — identify whether failures originate in the source system, the ETL process, or the validation rules themselves.
- Reprocessing — once a source system bug is fixed, corrected rows can be reloaded without a full historical backfill.
- Stakeholder transparency — the marketing team can see exactly how many records were rejected and why, rebuilding trust in the numbers that appear on dashboards.

5. Layer 3 — Natural Language BI Chatbot

5.1 Theory & Design Rationale

The bottleneck is not data access — the data exists and is clean. The bottleneck is the interface. SQL is a barrier to entry for non-technical users, and routing every question through the BI team creates latency that slows decision-making. The solution is a semantic layer — an abstraction between the raw query language and the human user.

BigQuery Data Agents represent Google's production-grade implementation of this pattern. Rather than building a custom application, the Data Agent is a fully managed service inside BigQuery that connects directly to the warehouse, uses Gemini to interpret questions, generates and executes SQL, and returns results in plain English — all within the GCP security boundary.

5.2 How BigQuery Data Agents Work

4. The user submits a natural language question through the agent interface.
5. The agent retrieves schema context — column names, types, and descriptions — from the connected BigQuery table. Column-level DDL descriptions are the agent's primary grounding for understanding what the data means.
6. The schema context and question are assembled into a prompt and sent to Gemini, which generates a BigQuery SQL query.
7. The agent executes the SQL using the project's service account credentials.
8. Results are returned to Gemini, which composes a plain English answer incorporating actual result values.

5.3 Why Column Descriptions Matter

The BigQuery DDL for gold.orders includes detailed OPTIONS(description=...) for every column. This is not documentation for humans alone — it is context that the Data Agent reads to understand each field. A column named revenue with description 'Final revenue in USD after discount — quantity x unit_price x (1 - discount_pct)' is far more actionable for the agent than a bare column name. Column descriptions directly influence the quality of generated SQL.

Agent System Instructions

You are a helpful BI assistant for an e-commerce company selling home and outdoor products across 5 US online stores.

order_date and ship_date are DATE columns. revenue is the final sale amount in USD after discount.

store_name canonical values: Garden Grove Store, Pacific Coast Store, Lakeside Outlet, Mountain View Store, Prairie Home Store.

order_status: Completed, Returned, Cancelled. Always include actual numbers. Respond in 2-3 plain English sentences.

6. Production & Scaling Considerations

6.1 The Gap Between POC and Production

The pipeline was built within GCP Free Tier constraints — BigQuery Sandbox, Cloud Functions, and the free Gemini API. These constraints required pragmatic choices appropriate for a proof of concept but not acceptable in a production environment. This section describes the production-grade alternatives and the engineering rationale for each upgrade.

6.2 Ingestion — Production Upgrades

Email automation via Gmail API + Pub/Sub

The current design requires a person to upload the CSV. In production, a Cloud Function subscribed to a Gmail Pub/Sub push notification would detect the email from the source system, extract the attachment automatically, and deposit it in GCS — eliminating the remaining manual step.

BigQuery Load Jobs over Streaming Inserts

Streaming inserts cost $0.01 per 200MB and are designed for real-time sub-second latency. A daily batch file has no such requirement. BigQuery Load Jobs from GCS are free, support larger files, and are the correct tool for batch ingestion.

Cloud Dataflow at high volume

At 10M+ rows per daily file, Cloud Functions hit memory and timeout limits. Cloud Dataflow provides horizontal autoscaling, exactly-once processing guarantees, and dead letter queue handling for failed rows.

6.3 Data Quality — Production Upgrades

dbt Core as the transformation framework

The BigQuery Stored Procedure lacks version control, automated lineage graphs, declarative schema tests, documentation generation, and CI/CD integration. dbt Core provides all of these on top of BigQuery SQL, decomposing the stored procedure into independently tested and documented models.

Great Expectations for richer data contracts

dbt tests cover structural constraints. Great Expectations extends this to statistical checks — verifying revenue distributions, detecting sudden spikes in return rates, validating email regex patterns — the classes of issues structural checks miss.

Real-time alerting via Pub/Sub

The dq_run_log is a passive record requiring manual querying. FAIL entries should trigger active Pub/Sub notifications routing to Slack or email within minutes of pipeline completion.

6.4 Chatbot — Production Upgrades

Identity-Aware Proxy for access control

The current agent is accessible to anyone with the link. IAP enforces corporate Google Workspace authentication at the infrastructure level before the agent responds.

Conversation memory via Cloud Firestore

Session state stored in Firestore enables multi-turn dialogue — users can ask follow-up questions referencing previous answers, mirroring how a human analyst would respond.

Query result caching via Cloud Memorystore

Common questions asked multiple times daily can be served from Redis cache in 100ms instead of triggering a new Gemini call and BigQuery query each time — reducing both cost and latency.

6.5 Pipeline Orchestration — Cloud Composer

In production, all three layers should be orchestrated as a single observable workflow. Cloud Composer (managed Apache Airflow) provides DAG-based orchestration with dependency management, retry logic, SLA monitoring, and a visual health dashboard.

Production DAG

DAG: daily_ecommerce_pipeline | Schedule: 0 6 * * *

check_gcs_arrived >> trigger_function >> validate_raw >> run_gold_pipeline >> check_dq_log >> alert_if_failed

Each step: retry_count=3, timeout=30m, on_failure_callback=notify_team

6.6 Cost Comparison

Service	POC (Free Tier)	Production (1M rows/day)	Cost Driver
BigQuery storage	10 GB free	~$20/month per TB	Historical data volume
BigQuery queries	1 TB/month free	~$5/month	95% reduction via partitioning
Cloud Functions	2M calls free	~$0.40/month	Execution time per invocation
Dataflow	Not used	~$50/month	Worker hours for large batches
Gemini / Vertex AI	Free tier	~$0.075 per 1M tokens	Number of agent questions
Cloud Composer	Not free	~$300/month	Cluster always running
Cloud Memorystore	Not free	~$50/month	Redis instance size

Single Biggest Cost Optimisation

BigQuery partitioning and clustering. A well-partitioned 1TB table reduces query costs from $5 per full scan to $0.25 per partitioned query — a 95% reduction that compounds across every dashboard refresh, scheduled query, and agent question. Designed in at the raw layer (partition by load date) and gold layer (partition by order_date, cluster by store_name and category).